Airbnb NYC Market Segmentation and Price Prediction - Group D

Members: Members: Mattia Russo, Vinod Jadhav, Alixandria Ali, Fiaz Ali, Ramon Flores Arteaga

Business question 1
Business Problem and Objective 1
Dataset and Preparation 1
Exploratory Findings 2
Spatial Distribution of Listings and Prices 3
Regression: Price Prediction 5
Clustering: Market Segments 7
Effect of the Classical ML Approach 8
Business Recommendations 8
Conclusion 9
References 9
Appendices 10
Appendix 1 - Exploratory Data Analysis Code 10
Appendix 2 - Clustering Data Preparation 11
Appendix 3 - Cluster Selection 12
Appendix 4 - Regression Modelling and Evaluation 13
Appendix 5 - Cluster Selection Diagnostics 15

Business question

What market segments exist in the New York City Airbnb market, and how can pricing prediction help hosts develop more competitive pricing strategies?

Business Problem and Objective

The New York City Airbnb market contains substantial differences in price across room types and locations. These differences suggest that pricing guidance based on broad city-wide comparisons may overlook important differences between listings. The analysis therefore combines clustering and regression. Clustering is used to identify groups of listings with similar characteristics, while regression tests how accurately price can be predicted and which listing characteristics contribute most to prediction.

Dataset and Preparation

The AB_NYC_2019 dataset contains 48,895 listings and 16 variables covering price, room type, location, minimum-night requirements, reviews, host listing activity and availability. (Dgomonov, 2019) Exploratory analysis found a strongly right-skewed price distribution: the mean price was $152.72, the median was $106, and 95% of listings were priced at $355 or below. The 10,052 listings with no last_review all had zero reviews, indicating that they had not received reviews. For clustering, missing reviews_per_month values were set to zero, numerical variables were standardised, and extreme

minimum_nights values were capped at 365 to limit their influence without removing plausible long-term listings.

Exploratory Findings

Room type and location showed the clearest pricing differences. Entire homes/apartments had the highest median price at $160, followed by private rooms at $70 and shared rooms at $45. Manhattan also had the highest median price at $150, compared with $90 in Brooklyn, $75 in Queens and Staten Island, and $65 in the Bronx. (Dgomonov, 2019)

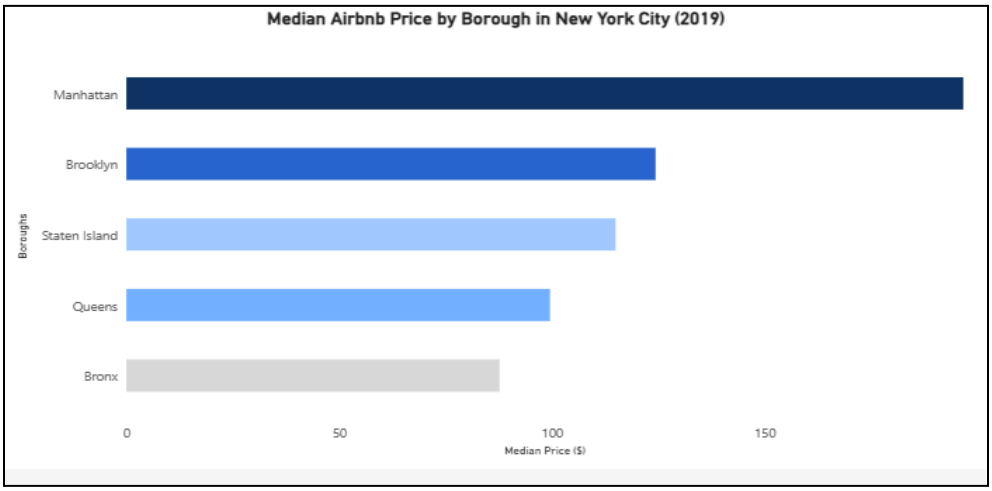


Figure 1a. Median Airbnb Price by Borough in New York City (2019)

Source: Authors' analysis of Dgomonov (2019).

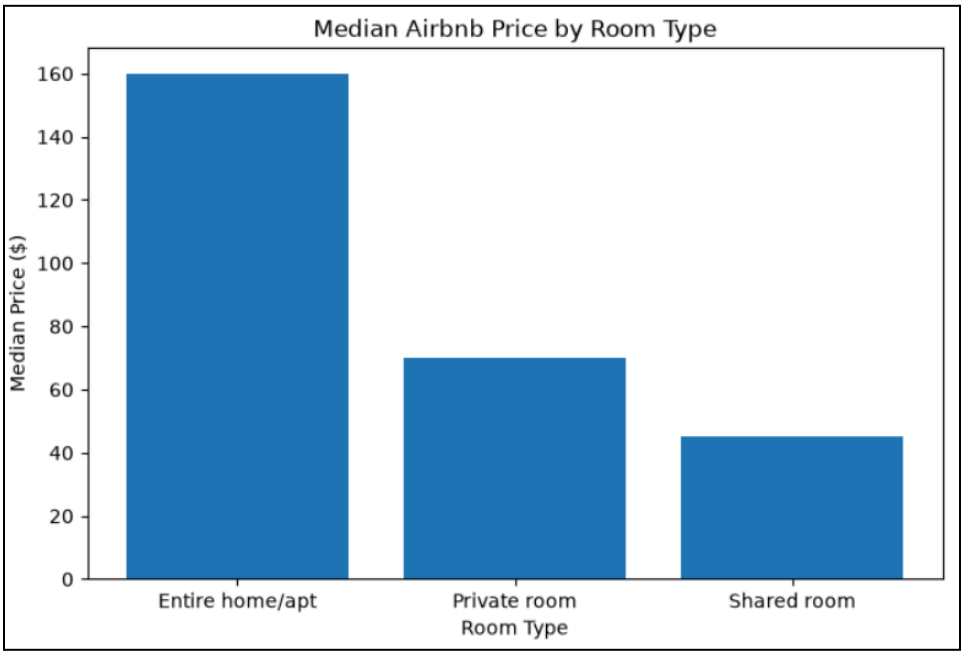


Figure 1b. Median Airbnb Price by Room Type in New York City (2019)

Source: Authors' analysis of Dgomonov (2019).

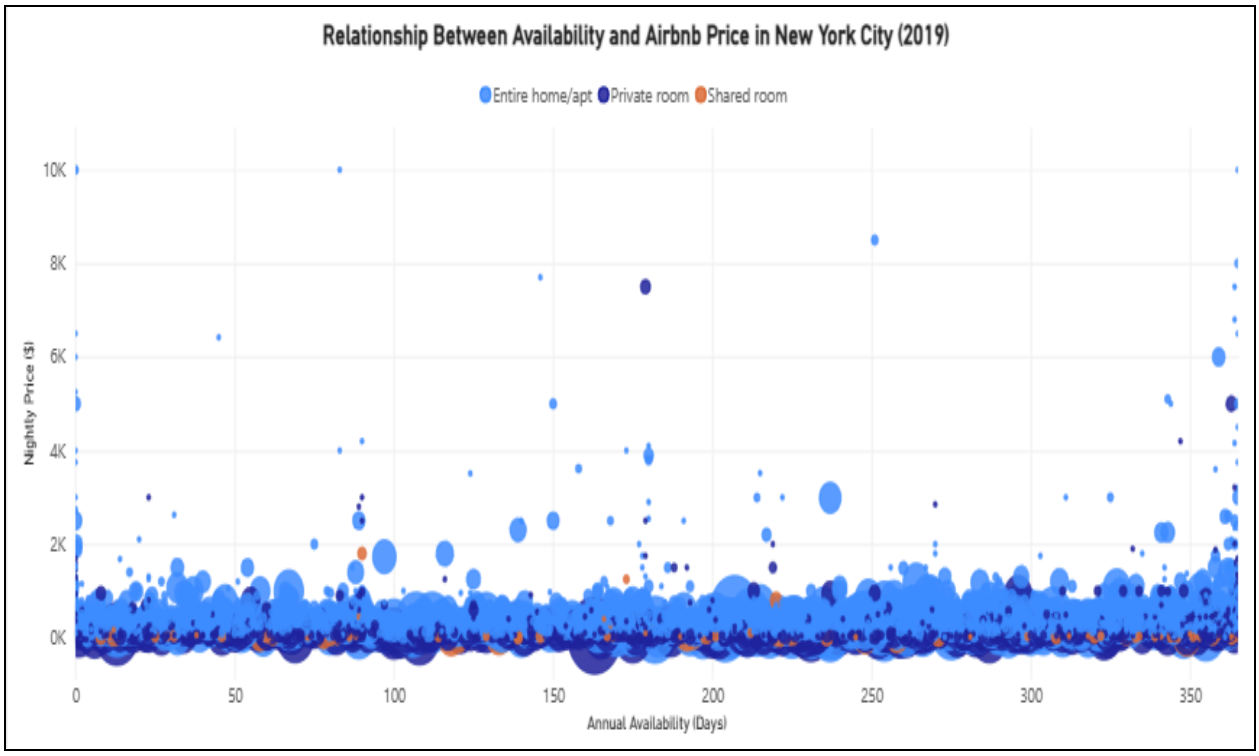


Figure 2. Relationship Between Availability and Airbnb Price in New York City (2019)
Source: Authors' analysis of Dgomonov (2019).

Spatial Distribution of Listings and Prices



Figure 3a: Airbnb Listing Density and Average Price Across Queens, New York City (2019)
Source: Authors' analysis of Dgomonov (2019).



Figure 3b: Airbnb Listing Density and Average Price Across Manhattan, New York City (2019)
Source: Authors' analysis of Dgomonov (2019).

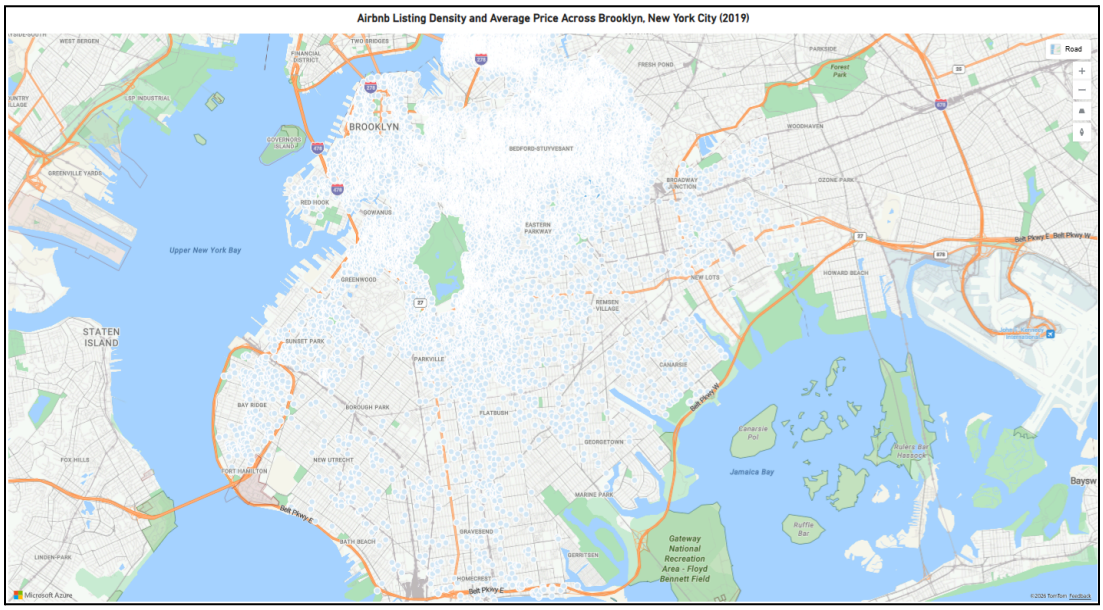


Figure 3c: Airbnb Listing Density and Average Price Across Brooklyn, New York City (2019)
Source: Authors' analysis of Dgomonov (2019).

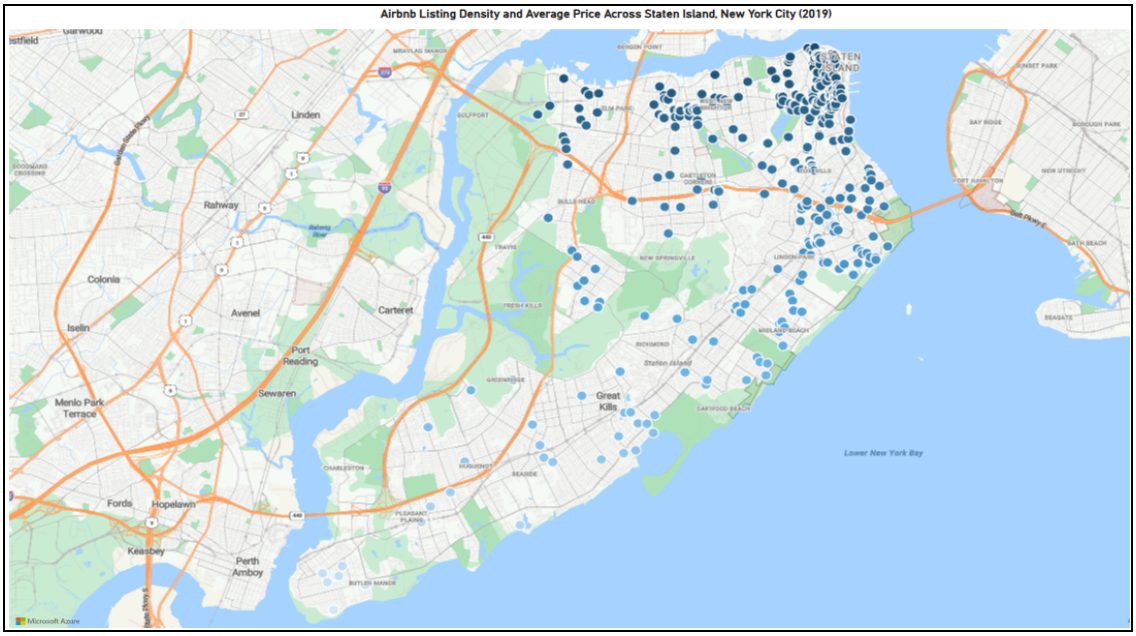


Figure 3d: Airbnb Listing Density and Average Price Across Staten Island, New York City (2019)
Source: Authors' analysis of Dgomonov (2019).

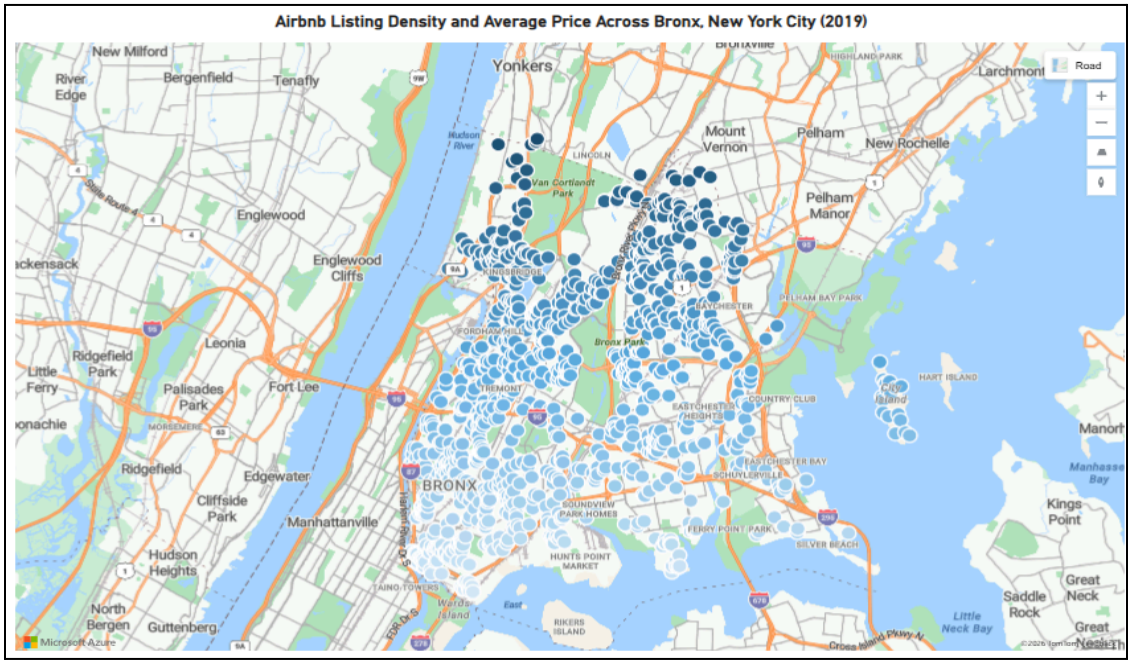


Figure 3e: Airbnb Listing Density and Average Price Across Bronx, New York City (2019)
Source: Authors' analysis of Dgomonov (2019).

Regression: Price Prediction

Four approaches were evaluated: a median-price baseline, Ridge regression, Random Forest and Histogram Gradient Boosting. Eleven zero-price listings were excluded. The remaining data were split by

host into 39,140 training and 9,744 test listings, ensuring that no host appeared in both sets. Five-fold grouped cross-validation was used for model selection.

Histogram Gradient Boosting achieved the best overall results, with a cross-validation MAE of $58.69, test MAE of $59.38, RMSE of $205.05 and R^2 of 0.146. Although Random Forest produced a marginally lower test MAE of $59.24, Gradient Boosting performed slightly better across the other evaluation measures. Its test MAE was 30.96% lower than the median baseline. However, the R^2 indicates that it explained only 14.6% of price variation, reflecting the difficulty of predicting Airbnb prices using the available variables.

Test-set permutation importance identified room type as the strongest predictor: shuffling it increased MAE by $27.54. Neighbourhood followed at $11.89, availability_365 at $6.70 and minimum_nights at $4.51. Extreme prices substantially affected performance: only 88 test listings, representing 0.9% of the test set, accounted for 82.73% of the total squared prediction error.

Table 1. Regression Model Performance

Model	*CV MAE*	*Test MAE*	*Test RMSE*	*Test R^2*
Median baseline	—	$86.01	$227.48	-0.051
Ridge regression	$60.90	$61.87	$208.84	0.114
Random Forest	$58.81	$59.24	$205.44	0.143
Histogram Gradient Boosting	$58.69	$59.38	$205.05	0.146

Source: Authors' analysis of Dgomonov (2019).

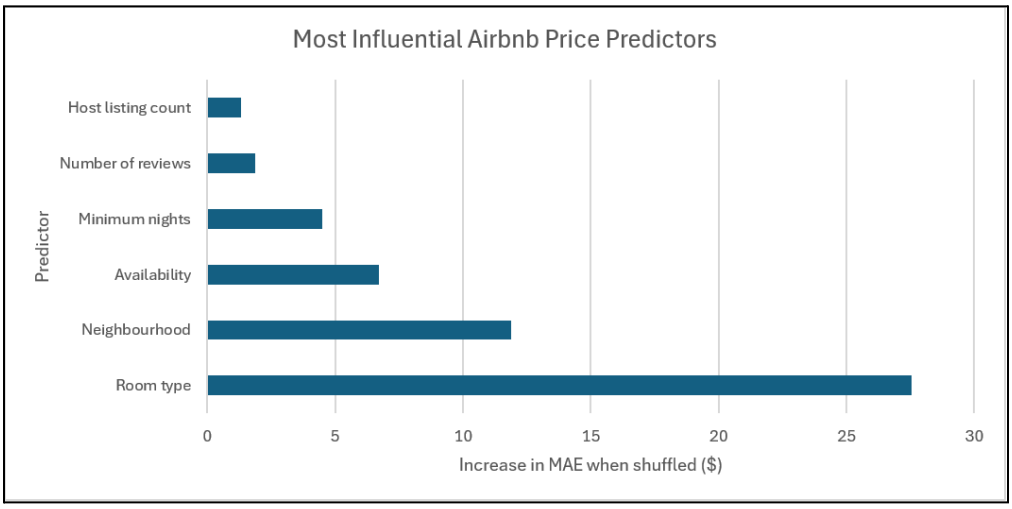


Figure 4. Most Influential Price Predictors
Source: Authors' analysis of Dgomonov (2019).

Clustering: Market Segments

K-Prototypes were used because the clustering combined numerical variables with the categorical neighbourhood_group variable. Price was deliberately excluded so that segments were formed from listing characteristics rather than price itself. Clustering was performed separately for entire homes/apartments, private rooms and shared rooms.

For the Entire home/apartment group, the Elbow Method did not show a single clear optimal value of k, so the Silhouette Score using Gower distance was also considered. The scores were 0.3094 for k=4 and 0.3164 for k=9. Although k=9 was numerically higher, the difference was small, so k=4 was retained as a simpler, interpretable solution for the pricing objective. This produced four entire-home clusters, three private-room clusters and two shared-room clusters, giving nine market segments overall.

The cluster profiles showed differences within each room type. Entire_2 was a small segment of 551 listings with the highest median price ($252), a median minimum stay of 29 nights and 99.6% of listings in Manhattan. Entire_3 had the same $150 median as Entire_0 but much higher review activity. Private-room medians were close to $70, although review activity and availability differed sharply. Shared_0 had a $55 median price, while Shared_1 was lower at $39 and had median availability of 347 days.

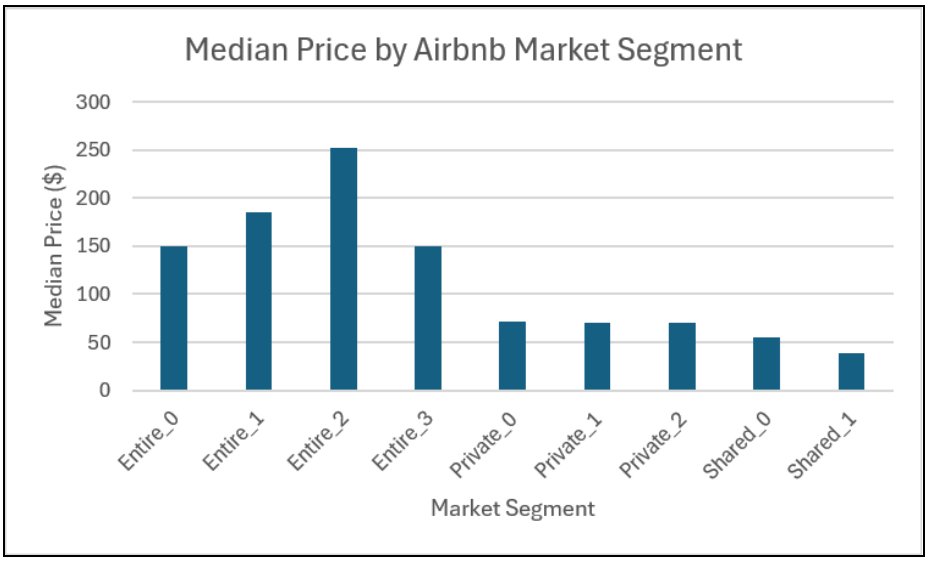


Figure 5. Median Price by Airbnb Market Segment
Source: Authors' analysis of Dgomonov (2019).

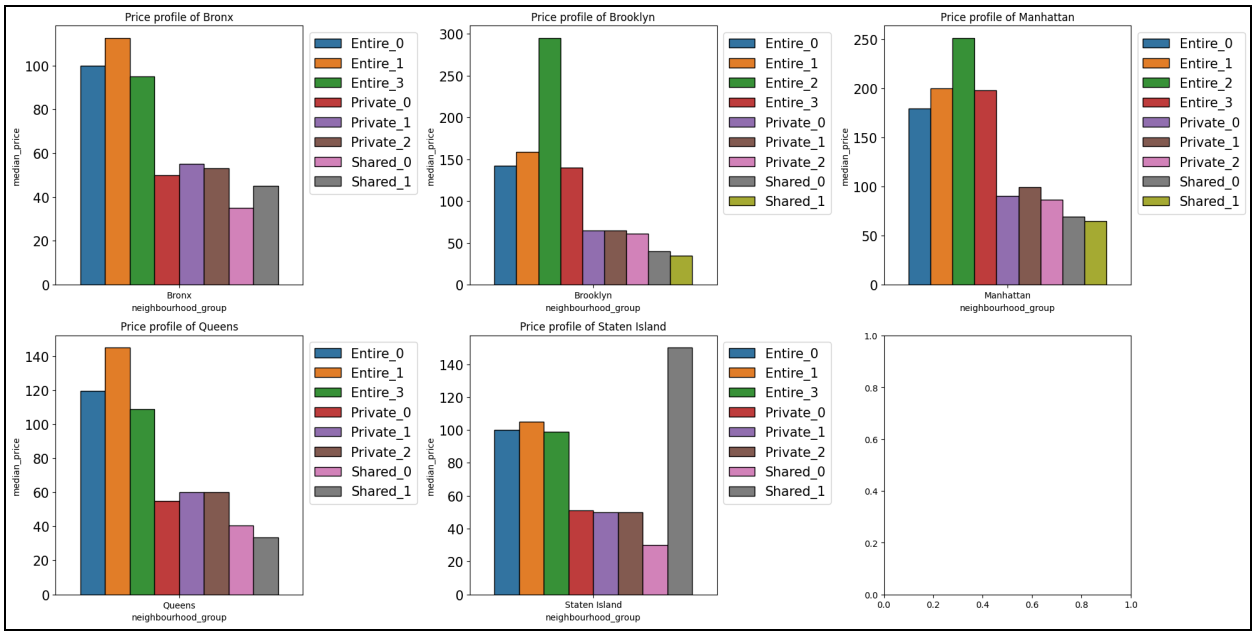


Figure 6. Median Airbnb Price by Room Type / Neighbourhood
Source: Authors' analysis of Dgomonov (2019).

Effect of the Classical ML Approach

Regression and clustering answered different parts of the business question. Regression quantified price-prediction performance and identified which available variables contributed most to prediction. Clustering identified market groups without using price to form them, allowing prices to be compared after the segments were established. The results are limited to the structured information available in AB_NYC_2019: the regression retained substantial unexplained price variation, and the clustering describes listing-based market segments rather than guest profiles.

Business Recommendations

The recommended strategy is a two-stage pricing approach. Airbnb can assign each listing to a comparable market segment using room type, neighbourhood, availability, minimum-stay requirements, review activity and host listing count. The regression estimate can then be presented alongside the segment median, allowing hosts to assess whether their price is consistent with comparable properties rather than the NYC market.

Implementation could begin with a controlled pilot comparing segment-based guidance with existing pricing recommendations. Success could be evaluated through host adoption, booking conversion, occupancy and revenue per available night. Extreme-price listings require separate treatment because 0.9% of the test sample generated 82.73% of total squared prediction error. Regular retraining with additional variables, including amenities, property quality, seasonality and proximity to attractions, could address limitations in the 2019 dataset.

Conclusion

Histogram Gradient Boosting produced the strongest performance and reduced test MAE by 30.96% relative to the median baseline. However, its R^2 of 0.146 indicates that most price variation remained unexplained. Prediction is therefore better suited to supporting, rather than automating, pricing decisions. Combining model estimates with segment-specific comparisons offers a more defensible strategy by recognising market differences, improving on a city-wide benchmark and allowing hosts to incorporate property-specific information not captured by the model.

References

Dgomonov (2019) *New York City Airbnb open data* [Dataset]. Kaggle. Available at: https://www.kaggle.com/dgomonov/new-york-city-airbnb-open-data (Accessed: 29 August 2026).

Word count: 1009 Words (excluding images, list of references, tables, appendices).

Appendices

Appendix 1 - Exploratory Data Analysis Code

Code extracts showing the focused exploratory analysis of price, room type and neighbourhood/location.

A1.1 Load dataset

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("../data/raw/AB_NYC_2019.csv")

print("Dataset loaded successfully.")
print("Dataset shape:", df.shape)
```

A1.2 Price distribution and 95th-percentile view

```
df["price"].describe().round(2)
median_price = df["price"].median()
price_95 = df["price"].quantile(0.95)
price_below_95 = df[df["price"] <= price_95]

plt.figure(figsize=(8, 5))
plt.hist(df["price"], bins=50)
plt.title("Distribution of Airbnb Listing Prices")
plt.xlabel("Price ($)")
plt.ylabel("Number of Listings")
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(price_below_95["price"], bins=40)
plt.title("Airbnb Price Distribution – Up to 95th Percentile")
plt.xlabel("Price ($)")
plt.ylabel("Number of Listings")
plt.show()
```

A1.3 Room-type analysis.

```
# Room Type analysis

df["room_type"].value_counts()
df.groupby("room_type")["price"].agg(
    ["count", "mean", "median", "min", "max"]
).round(2)

median_price_by_room = (
    df.groupby("room_type")["price"]
      .median()
      .sort_values(ascending=False)
)

plt.figure(figsize=(8, 5))
plt.bar(median_price_by_room.index, median_price_by_room.values)
plt.title("Median Airbnb Price by Room Type")
plt.xlabel("Room Type")
plt.ylabel("Median Price ($)")
plt.show()
```

A1.4 Neighborhood/location analysis

```
df["neighbourhood_group"].value_counts()
df.groupby("neighbourhood_group")["price"].agg(
    ["count", "mean", "median", "min", "max"]
).round(2)

median_price_by_neighbourhood_group = (
    df.groupby("neighbourhood_group")["price"]
      .median()
      .sort_values(ascending=False)
)
plt.figure(figsize=(8, 5))
plt.bar(
    median_price_by_neighbourhood_group.index,
    median_price_by_neighbourhood_group.values
)
plt.title("Median Airbnb Price by Neighbourhood Group")
plt.xlabel("Neighbourhood Group")
plt.ylabel("Median Price ($)")
plt.xticks(rotation=20)
plt.show()
print("Number of unique neighbourhoods:", df["neighbourhood"].nunique())
df["neighbourhood"].value_counts().head(10)
```

Appendix 2 - Clustering Data Preparation

Code showing the selected numerical/categorical variables, StandardScaler, and preparation for K-Prototypes.

Features' Selection:

```python
df_clstr = df_cluster.drop(columns = ['price', 'name', 'host_id', 'host_name', 'last_review','neighbourhood','latitude','longitude']).set_index('id')
df_clstr
```

Split Dataset by Room Type:

```python
df_entire = df_clstr[df_clstr['room_type'] == 'Entire home/apt'].copy()
df_private = df_clstr[df_clstr['room_type'] == 'Private room'].copy()
df_shared = df_clstr[df_clstr['room_type'] == 'Shared room'].copy()
```

```python
cluster_df = df_entire.drop(columns=['room_type']).copy() # we remove a redunant column
```

Identification of Numerical and Categorical Variables:

```python
# we separate numeric column from categorical

numeric = cluster_df.select_dtypes(include = ['int','float'])

categorical = cluster_df.select_dtypes(include = ['object','category'])

print('Numeric:', list(numeric.columns))
print('categorical:', list(categorical.columns))
```

```python
X = cluster_df[list(numeric.columns) + list(categorical.columns)].copy()
X.head()
```

Scaling Numerical Features:

```python
scaler = StandardScaler()
X[numeric.columns] = scaler.fit_transform(X[numeric.columns])
```

Identification of the position of Categorical Columns for K-prototypes:

```python
cat_index = list(range(len(numeric.columns), len(numeric.columns) + len(categorical.columns)))
print('Cat Index:', cat_index)
```

Appendix 3 - Cluster Selection

Code used to calculate Gower distance and silhouette scores for candidate values of k

Import of main libraries

```python
from sklearn.metrics import silhouette_score
import gower
```

Silhouette scores calculated using Gower distance to identify best k

```python
# Gower distance matrix calculated on the sample
gower_dist_matrix1 = gower.gower_matrix(X_sample)

silhouette_scores1 = []

K_range = range(2, 10)

for k in K_range:

    kp = KPrototypes(
        n_clusters=k,
        init='Cao',
        random_state=42,
        n_init=5,
        verbose=0
    )

    # Fit K-Prototypes on the full dataset
    kp.fit(X, categorical=cat_index)

    # Predict cluster membership for the sample
    sample_clusters = kp.predict(
        X_sample,
        categorical=cat_index
    )

    # Calculate Silhouette Score using Gower distance
    score = silhouette_score(
        gower_dist_matrix1,
        sample_clusters,
        metric='precomputed'
    )

    silhouette_scores1.append(score)

    print(f'k={k} -> Silhouette Score: {score:.4f}')
```

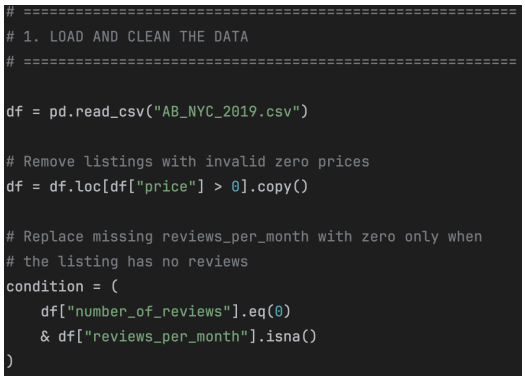
K-prototype implementation

```python
k_optimal = 4

kp_entire = KPrototypes(n_clusters=k_optimal, init='Cao', random_state=42, n_init=5)
clusters = kp_entire.fit_predict(X, categorical=cat_index)
```

Appendix 4 - Regression Modelling and Evaluation

Code extract showing the regression model fitting/evaluation, ideally the model-comparison section.
Cleaning and loading Data.



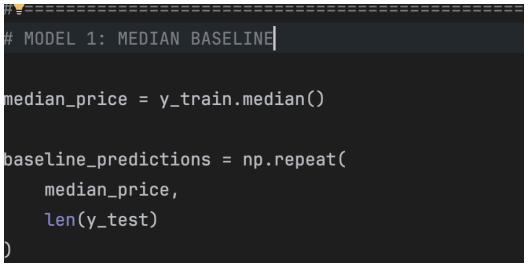
```
# ============================================================
# 1. LOAD AND CLEAN THE DATA
# ============================================================

df = pd.read_csv("AB_NYC_2019.csv")

# Remove listings with invalid zero prices
df = df.loc[df["price"] > 0].copy()

# Replace missing reviews_per_month with zero only when
# the listing has no reviews
condition = (
    df["number_of_reviews"].eq(0)
    & df["reviews_per_month"].isna()
)
```

Model #1 Base line



```
# ============================================================
# MODEL 1: MEDIAN BASELINE

median_price = y_train.median()

baseline_predictions = np.repeat(
    median_price,
    len(y_test)
)
```

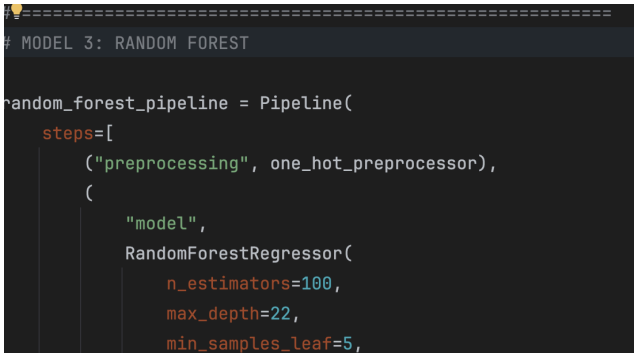
Model #2 Ridge Regression



```
# ============================================================
# MODEL 2: RIDGE REGRESSION

ridge_pipeline = Pipeline(
    steps=[
        ("preprocessing", one_hot_preprocessor),
        ("model", Ridge(alpha=10.0))
    ]
)
```

Model #3 Random Forest



```
# ============================================================
# MODEL 3: RANDOM FOREST

random_forest_pipeline = Pipeline(
    steps=[
        ("preprocessing", one_hot_preprocessor),
        (
            "model",
            RandomForestRegressor(
                n_estimators=100,
                max_depth=22,
                min_samples_leaf=5,
```

Model #4 Histogram Gradient Boosting

```
# ============================================================
# MODEL 4: HISTOGRAM GRADIENT BOOSTING

histogram_pipeline = Pipeline(
    steps=[
        ("preprocessing", ordinal_preprocessor),
        (
            "model",
            HistGradientBoostingRegressor(
                categorical_features=categorical_mask,
                learning_rate=0.08,
                max_iter=250,
                max_leaf_nodes=31,
                l2_regularization=1.0,
                random_state=42
```

Evaluating median baseline

```
# Evaluate the median baseline
results = []

baseline_metrics = calculate_metrics(
    y_test,
    baseline_predictions
)

results.append({
    "Model": "Median baseline",
    "CV MAE": np.nan,
    **baseline_metrics
})
```

Grouped cross validation and model training

```
# ============================================================
## GROUPED CROSS-VALIDATION AND MODEL TRAINING
grouped_cv = GroupKFold(n_splits=5)

models = {
    "Ridge regression": ridge_model,
    "Random forest": random_forest_model,
    "Histogram gradient boosting": histogram_model
}

for model_name, model in models.items():
    cv_scores = -cross_val_score(
        model,
        X_train,
        y_train,
        groups=train_groups,
        cv=grouped_cv,
        scoring="neg_mean_absolute_error",
        n_jobs=1
    )
```

Appendix 5 - Cluster Selection Diagnostics

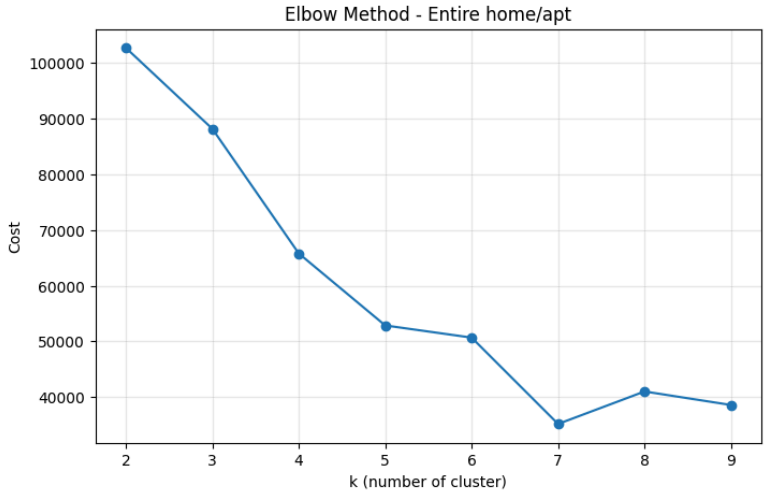
Elbow and/or silhouette plots for the room types, especially the Entire home/apartment silhouette plot
Elbow method Implementation and Plot:

```python
costs = []
K_range = range(2, 10)

for k in K_range:
    kp = KPrototypes(n_clusters=k, init='Cao', random_state=42, n_init=1, verbose=0)
    kp.fit_predict(X, categorical=cat_index)
    costs.append(kp.cost_)
    print(f"Entire home - k={k} -> cost={kp.cost_:.2f}")

plt.figure(figsize=(8, 5))
plt.plot(list(K_range), costs, marker='o')
plt.xlabel('k (number of cluster)')
plt.ylabel('Cost')
plt.title('Elbow Method - Entire home/apt')
plt.grid(True, alpha=0.3)
plt.show()
```

Python



Plot of Silhouette score:

```python
sns.lineplot(x=K_range, y=silhouette_scores1, marker='o')

plt.title('Silhouette Score v K')
plt.xlabel('K')
plt.ylabel('Silhouette score')
plt.show()
```

Python

